

# 同步请求下载记录的try{}catch异常处理：场景、实现与最佳实践

在数据交互场景中，“获取用户下载记录”是高频需求，而同步请求（如uni-app的 `uni.requestSync`、Node.js的同步HTTP请求）虽能简化代码执行流程，但一旦出现异常（网络中断、接口报错、数据格式错误等），极易导致程序阻塞或崩溃。`try{}catch` 作为JavaScript的核心异常捕获机制，能精准拦截同步请求中的各类异常，结合业务场景设计合理的处理逻辑，是保障功能稳定的关键。本文将从异常类型分析、分层处理实现、最佳实践三个维度，完整解析同步请求下载记录的异常处理方案。

## 一、同步请求下载记录的核心异常类型

同步请求下载记录的全链路中，异常可能出现在“请求发起”“接口响应”“数据解析”“本地处理”四个阶段，不同阶段的异常成因与表现差异显著，需针对性处理：

| 异常阶段   | 典型异常场景                    | 异常特征                                     | 是否可被try{}catch捕获    |
|--------|---------------------------|------------------------------------------|---------------------|
| 请求发起阶段 | 网络中断、域名解析失败、跨域限制          | 同步请求直接阻塞，抛出“网络错误”类异常                     | 是                   |
| 接口响应阶段 | 接口超时、服务器500错误、401未授权      | 返回异常状态码（非200），响应体无有效数据                   | 需手动判断状态码<br>异常可直接捕获 |
| 数据解析阶段 | 响应体格式错误（非JSON）、字段缺失、类型不匹配 | 执行JSON.parse时抛出“语法错误”，或访问字段时报“undefined” | 是                   |
| 本地处理阶段 | 本地缓存写入失败、下载记录格式化错误        | 执行本地存储或数据处理方法时抛出异常                       | 是                   |



注意：同步请求的“接口超时”异常在不同环境下表现不同——uni-app的 `uni.requestSync` 超时会直接抛出可捕获异常，而原生JavaScript的同步XMLHttpRequest超时则需通过onerror事件处理，需额外适配。

## 二、分层式异常处理实现（结合uni-app场景）

针对下载记录同步请求的全链路异常，采用“分层捕获+精准处理”策略：在 `try` 块中执行完整业务流程（请求发起→响应处理→数据解析→本地存储），在 `catch` 块中分类处理不同类型异常，同时补充“接口响应状态码校验”作为异常处理的补充（非200状态码不触发catch，但属于业务异常）。

## 1. 核心代码实现（同步请求+异常处理）

Code block

```
1  // services/downloadRecordService.js
2  import { getStorageSync, setStorageSync } from '@utils/storage.js';
3  import { showToast, showModal } from '@utils/feedback.js';
4
5  /**
6   * 同步请求用户下载记录（核心方法）
7   * @param {Object} params - 请求参数 (userId: 用户ID, page: 页码, size: 每页条数)
8   * @param {string} token - 用户登录令牌
9   * @returns {Object} 处理结果 (success: 是否成功, data: 下载记录列表, msg: 提示信息)
10  */
11  export const syncDownloadRecords = (params, token) => {
12    // 初始化返回结果
13    const result = {
14      success: false,
15      data: [],
16      msg: ''
17    };
18
19    try {
20      // 1. 前置校验：参数与令牌校验（避免无效请求）
21      if (!params.userId || !token) {
22        throw new Error('参数异常：用户ID或令牌缺失');
23      }
24
25      // 2. 同步发起请求（uni-app同步请求API）
26      const requestResult = uni.requestSync({
27        url: 'https://api.example.com/user/downloadRecords',
28        method: 'GET',
29        data: params,
30        header: {
31          'Authorization': `Bearer ${token}`,
32          'Content-Type': 'application/json'
33        },
34        timeout: 10000 // 10秒超时
35      });
36
37      // 3. 响应状态码校验（非200属于业务异常，手动抛出）
38      if (requestResult.statusCode !== 200) {
39        const errorMsg = getStatusCodeMsg(requestResult.statusCode);
```

```

40         throw new Error(`接口响应异常: ${errorMsg} (状态码:
${requestResult.statusCode}) `);
41     }
42
43     // 4. 数据解析 (同步请求的data已自动JSON.parse, 此处处理格式异常)
44     const responseData = requestResult.data;
45     if (!responseData || typeof responseData !== 'object') {
46         throw new Error('数据解析异常: 响应体格式非法');
47     }
48
49     // 5. 业务逻辑校验 (接口返回的code字段校验)
50     if (responseData.code !== 200) {
51         throw new Error(`业务异常: ${responseData.msg || '获取下载记录失败'}`);
52     }
53
54     // 6. 下载记录格式化 (处理字段缺失、类型转换)
55     const formattedRecords = formatDownloadRecords(responseData.data || []);
56
57     // 7. 本地缓存 (将同步后的记录存入本地, 提升下次访问速度)
58     setStorageSync('user_download_records', formattedRecords);
59     setStorageSync('download_records_update_time', Date.now()); // 记录更新时间
60
61     // 8. 处理成功: 更新返回结果
62     result.success = true;
63     result.data = formattedRecords;
64     result.msg = '下载记录同步成功';
65
66     } catch (error) {
67         // 9. 异常捕获与分类处理
68         result.msg = error.message || '下载记录同步失败, 请重试';
69         // 根据错误类型执行不同处理逻辑
70         handleSyncError(error, result);
71     }
72
73     return result;
74 };
75
76 /**
77  * 根据状态码获取对应提示信息
78  * @param {number} statusCode - HTTP状态码
79  * @returns {string} 状态码描述
80  */
81 const getStatusCodeMsg = (statusCode) => {
82     const statusMap = {
83         400: '请求参数错误',
84         401: '身份已过期, 请重新登录',
85         403: '无权限访问',

```

```
86      404: '接口不存在',
87      500: '服务器内部错误',
```

## 2. 页面中调用（结合用户交互）

在页面中调用同步方法时，通过加载状态控制避免重复请求，同时根据返回结果更新UI，确保用户体验流畅：

Code block

```
1  <script>
2  import { syncDownloadRecords } from '@services/downloadRecordService.js';
3  import { getStorageSync } from '@utils/storage.js';
4
5  export default {
6    data() {
7      return {
8        downloadRecords: [], // 下载记录列表
9        isLoading: false,    // 加载状态
10       isEmpty: false,      // 记录为空状态
11       errorMsg: ''         // 错误提示
12     };
13   },
14   onLoad() {
15     // 页面加载时同步下载记录
16     this.fetchDownloadRecords();
17   },
18   methods: {
19     /**
20      * 触发同步下载记录
21      * @param {boolean} isRefresh - 是否为下拉刷新
22      */
23     async fetchDownloadRecords(isRefresh = false) {
24       // 控制加载状态
25       this.isLoading = true;
26       this.errorMsg = '';
27
28       // 获取用户信息与令牌
29       const userInfo = getStorageSync('userInfo');
30       if (!userInfo) {
31         this.isLoading = false;
32         this.errorMsg = '请先登录以查看下载记录';
33         return;
34       }
35
36       // 调用同步方法（同步方法无需await，但可配合加载状态）
37       const result = syncDownloadRecords(
```

```

38         { userId: userInfo.id, page: 1, size: 20 },
39         userInfo.token
40     );
41
42     // 根据结果更新UI
43     this.isLoading = false;
44     if (isRefresh) {
45         uni.stopPullDownRefresh(); // 停止下拉刷新
46     }
47
48     if (result.success) {
49         this.downloadRecords = result.data;
50         this.isEmpty = result.data.length === 0;
51     } else {
52         this.errorMsg = result.msg;
53         // 若有本地缓存数据, 优先展示
54         if (result.data.length > 0) {
55             this.downloadRecords = result.data;
56             this.isEmpty = false;
57         }
58     }
59 },
60 /**
61  * 下拉刷新触发重新同步
62  */
63 onPullDownRefresh() {
64     this.fetchDownloadRecords(true);
65 },
66 /**
67  * 重试同步
68  */
69 handleRetry() {
70     this.fetchDownloadRecords();
71 }
72 }
73 };
74 </script>

```

### 3. 配套工具函数（反馈与存储）

异常处理中依赖的提示与存储工具函数，确保交互一致性：

Code block

```

1 // utils/feedback.js
2 /**
3  * 显示轻提示

```

```
4  * @param {string} title - 提示文本
5  * @param {string} type - 类型 (success/error/info)
6  */
7  export const showToast = (title, type = 'info') => {
8    const iconMap = {
9      success: 'success',
10     error: 'none',
11     info: 'none'
12   };
13   const bgColorMap = {
14     success: '#4cd964',
15     error: '#ff3b30',
16     info: '#323233'
17   };
18   uni.showToast({
19     title: title,
20     icon: iconMap[type],
21     duration: 2000,
22     image: type === 'error' ? '/static/icons/error.png' : '',
23     mask: true
24   });
25 };
26
27 /**
28  * 显示模态框
29  * @param {Object} options - 模态框配置
30  */
31 export const showModal = (options) => {
32   const defaultOptions = {
33     title: '提示',
34     content: '',
35     confirmText: '确认',
36     cancelText: '取消',
37     showCancel: true,
38     success: () => {}
39   };
40   uni.showModal({ ...defaultOptions, ...options });
41 };
42
43 // utils/storage.js
44 /**
45  * 同步获取本地缓存
46  * @param {string} key - 缓存键
47  * @returns {any} 缓存值
48  */
49 export const getStorageSync = (key) => {
50   try {
```

```

51     return uni.getStorageSync(key);
52 } catch (error) {
53     console.error(`获取缓存${key}失败:`, error);
54     return null;
55 }
56 };
57
58 /**
59  * 同步设置本地缓存
60  * @param {string} key - 缓存键
61  * @param {any} value - 缓存值
62  */
63 export const setStorageSync = (key, value) => {
64     try {
65         uni.setStorageSync(key, value);
66     } catch (error) {
67         console.error(`设置缓存${key}失败:`, error);
68         throw new Error(`本地缓存写入失败: ${error.message}`);
69     }
70 };

```

## 三、异常处理的关键设计思路

### 1. 前置校验减少无效异常

在 `try` 块开头添加“参数校验”“令牌校验”，避免因缺少必要参数导致的无效请求，减少异常触发频率。例如：校验用户ID与令牌是否存在，若缺失直接抛出“参数异常”，无需发起网络请求。

### 2. 状态码与业务码双重校验

同步请求的 `statusCode`（HTTP状态码）不触发 `catch`，需手动校验：

- 401状态码：身份过期，引导重新登录；
- 500/502/504状态码：服务器异常，提示用户稍后重试；
- 接口返回的 `code` 字段（业务码）：非200时抛出业务异常，如“权限不足”“数据不存在”。

### 3. 异常分类与差异化处理

在 `catch` 块中根据错误信息分类处理，提升用户体验：

1. **网络异常**：提示检查网络，同时加载本地缓存数据，保障功能可用性；
2. **身份过期**：通过模态框引导用户重新登录，而非仅显示错误提示；
3. **客户端异常**（参数错误、数据解析错误）：打印详细日志便于调试，给用户简洁提示；
4. **服务器异常**：提示“系统繁忙，请稍后重试”，避免暴露技术细节。

## 4. 本地缓存作为异常兜底

将同步成功的下载记录存入本地缓存，当网络异常或接口报错时，从缓存中读取历史数据，实现“离线可用”的效果，减少异常对用户的影响。例如：网络中断时，提示“已加载本地缓存”，并展示缓存的下载记录。

## 5. 异常信息的“用户友好化”

将技术化的异常信息转换为用户易懂的提示，避免直接展示“JSON.parse error”“401 Unauthorized”等专业术语。例如：将“401”转换为“身份已过期，请重新登录”。

# 四、同步请求vs异步请求的异常处理差异

下载记录请求也可采用异步方式（`uni.request`），其异常处理与同步请求存在核心差异，需明确区分：

| 对比维度   | 同步请求<br>( <code>uni.requestSync</code> ) | 异步请求（ <code>uni.request</code> ）                            |
|--------|------------------------------------------|-------------------------------------------------------------|
| 异常捕获方式 | <code>try{}catch</code> 直接捕获所有执行异常       | 通过fail回调处理请求异常， <code>try{}catch</code> 仅能捕获success回调中的执行异常 |
| 代码执行流程 | 阻塞式执行，需等待请求完成后再执行后续代码                    | 非阻塞式，请求发起后继续执行后续代码，结果通过回调返回                                 |
| 超时异常处理 | 直接抛出异常，可被catch捕获                         | 触发fail回调，需在回调中处理                                            |
| 适用场景   | 简单场景、需要严格按顺序执行的流程                        | 复杂场景、避免页面阻塞的交互（如列表加载）                                       |



建议：小程序或App的页面交互中，优先使用异步请求+Promise封装，避免同步请求导致的页面卡顿；仅在工具类、初始化流程等非交互场景使用同步请求。

## 五、最佳实践与避坑指南

### 1. 避免在try块中包含无关代码



try块应仅包含“可能抛出异常的核心业务代码”，避免将UI更新、日志打印等无关代码放入，确保异常捕获的精准性。例如：将 `showToast` 等提示方法放在try/catch之外或catch块中。

## 2. 手动抛出异常时携带上下文信息

使用 `throw new Error()` 手动抛出异常时，携带具体的上下文信息（如状态码、参数名），便于调试与分类处理。例如：`throw new Error(`接口响应异常：401未授权`)`，而非 `throw new Error('接口异常')`。

## 3. 日志打印分级

客户端异常（参数错误、缓存失败）打印详细日志，服务器异常（500错误）可上报至日志系统，便于问题定位。例如：通过 `console.error` 打印异常堆栈信息。

## 4. 避免同步请求阻塞UI

同步请求会阻塞JavaScript执行线程，若请求耗时较长（如10秒超时），会导致页面卡顿、无法交互。解决方案：

- 设置合理的超时时间（建议5-10秒）；
- 在请求期间显示加载动画，告知用户“正在同步”；
- 复杂场景下改用异步请求+`async/await`，提升页面响应性。

## 5. 测试覆盖全异常场景

开发阶段需针对性测试异常场景，确保处理逻辑生效：

1. 网络异常：关闭网络后发起请求，验证是否加载本地缓存；
2. 身份过期：使用无效令牌请求，验证是否引导重新登录；
3. 数据异常：构造非JSON格式的响应体，验证是否捕获解析异常；
4. 服务器异常：通过接口调试工具返回500状态码，验证提示信息是否准确。

## 六、总结

同步请求下载记录的try{}catch异常处理，核心是“全链路覆盖+差异化处理”——不仅要捕获请求、解析、存储等阶段的技术异常，还要通过状态码校验覆盖业务异常，同时结合本地缓存实现异常兜底。关键在于：

1. 前置校验减少无效请求，降低异常触发概率；
2. 状态码与业务码双重校验，避免遗漏业务异常；
3. 异常分类处理，结合场景提供友好提示与兜底方案；
4. 通过日志打印与测试覆盖，确保异常处理逻辑生效。

通过本文的方案，可有效提升同步请求下载记录功能的健壮性，减少程序崩溃风险，同时保障用户在异常场景下仍能获得可用的服务体验。

（注：文档部分内容可能由 AI 生成）